

실습 가이드 · Practical Exercise

Gilded Rose

Gemini CLI로 완성하는 AI 기반 QA 실전

레거시 코드를 분석 · 테스트 · 리팩토링 · 기능 추가한다 | P-C-T-F 프롬프트 · GEMINI.md · Mom Test

11

단계 (0~9 + 보너스)

Python 3

pytest · pytest-cov

P-C-T-F

Persona·Context·Task·Format

기술 스택: Python 3 · pytest · pytest-cov (+ coverage.py) · Gemini CLI

무엇을, 왜 하는가

이미 존재하는 “레거시 코드”를 Gemini CLI로 분석 · 테스트 · 리팩토링 · 기능 추가하는 QA 실습입니다. 각 단계는 P-C-T-F 프롬프트를 gemini 세션에 붙여넣고 산출물 파일을 저장하며 진행합니다.



분석한다

요구사항과 코드 품질을 Gemini로 재정리하고 SOLID·Code Smell을 진단



테스트한다

경계값·parametrize 테스트를 설계하고 Golden Master로 회귀를 방어



리팩토링한다

테스트 Green을 유지하며 매직 넘버 상수화·전략 패턴으로 개선



기능 추가한다

Item 클래스 수정 없이 Conjured 규칙을 추가하고 Green 확인

핵심 시험 과제 Item 클래스를 수정하지 않고 Conjured(매일 -2 · 기한 후 -4 · 최소 0) 규칙을 추가한 뒤 `pytest -v` Green 확인

11단계 로드맵 — 0단계 + 1~9단계 + 보너스



각 단계 = P-C-T-F 프롬프트 붙여넣기 → 산출물 파일 저장

P-C-T-F 프롬프트 구조와 컨텍스트 파일



Persona

응답자의 역할·관점을 지정한다 (예: 시니어 Python QA 엔지니어)



Context

기술 스택·프로젝트 상황을 준다 (Python 3, pytest, pytest-cov)



Task

해야 할 작업을 구체적으로 나열한다 (번호 목록으로)



Format

산출물 형식·저장 파일명을 지정한다 (예: Markdown, ~.md 저장)

공통

컨텍스트 & 파일 참조

- GEMINI.md — 프로젝트 루트에 두면 모든 프롬프트에 자동 로딩
- @파일명 — 코드·문서를 프롬프트에 첨부 (예: @gilded_rose.py)
- /memory refresh — 세션에서 GEMINI.md 재로딩

절대 제약 (Non-negotiable)

- Item 클래스 수정 금지
- quality 범위 0~50 (Sulfuras 제외)
- sellIn 음수 허용 (경계, 예외 아님)
- 경계값 필수: quality 0·1·49·50 / sellIn 0~-1

아이템 타입별 비즈니스 규칙 (6종)

● 일반 Normal

매일 `quality -1`

기한 후 `-2` · 최소 0

● Aged Brie

오래될수록 `+1`

기한 후 `+2` · 최대 50

● Backstage Pass

`sellIn < 11` `+2` · `< 6` `+3`

콘서트 후(기한 지남) 0

● Sulfuras

`quality 80` 고정

`sellIn` 불변 · 판매 안 됨

● Conjured ★ 신규

일반의 2배 감소 (`-2`)

기한 후 `-4` · 최소 0

● 경계 불변식

$0 \leq \text{quality} \leq 50$

(Sulfuras 제외) · `sellIn` 음수 OK

저장소 · 환경 · staging 브랜치

0단계

준비 4단계

0-A 저장소·환경 준비

clone · venv · pip install

0-B staging 브랜치

main → staging → 작업

0-C VS Code + Gemini 연동

0-D GEMINI.md 초안 생성

0-A · 환경 준비

```
cd C:\DEV
git clone <저장소 주소> gilded-rose
cd gilded-rose
code .
python -m venv .venv
.venv\Scripts\activate
# macOS/Linux:
# source .venv/bin/activate
pip install pytest pytest-cov

# 상태 확인
dir
pytest -q
```

0-B · staging 브랜치

```
# 통합 브랜치
git switch -c staging

# QA 작업 브랜치
git switch -c \
    feature/qa-prompting

git add .
git commit -m \
    "chore: QA 브랜치 초기화"

# 이후 VS Code에서
gemini
```

프로젝트 전역 규칙을 컨텍스트 파일로

▶ Gemini 프롬프트

[P] 당신은 레거시 코드 QA/리팩토링을 돕는 시니어 Python 엔지니어입니다.

[C] Gilded Rose 프로젝트에서 Gemini CLI가 항상 따라야 할 규칙을 GEMINI.md로 작성한다.

[T] 아래를 만족하는 GEMINI.md 내용을 작성해줘.

- 기술 스택: Python 3 + pytest + pytest-cov
- 절대 규칙: Item 수정 금지, quality 0~50, sellIn 음수 허용
- 테스트: Given-When-Then, parametrize, 경계값(quality 0/50, sellIn 0/-1) 포함
- 리팩토링: 테스트 Green 상태에서만 진행, 매직 넘버 상수화 권장
- Python: type hint 권장, dataclass 활용 가능

[F] GEMINI.md 에 붙여넣을 Markdown 텍스트

개념

GEMINI.md 란?

- 프로젝트 루트에 두는 전역 규칙 파일
- 모든 프롬프트에 자동으로 로딩된다
- 기술 스택·제약·테스트·리팩토링 규칙을 한 곳에

TIP

저장 & 재로딩

파일 저장 후 세션에서:

/memory refresh

→ GEMINI.md를 다시 읽어 규칙을 적용

Requirements Analysis → requirements_analysis.md

▶ Gemini 프롬프트

```
@GildedRoseRequirements.txt @README.md
[P] 시니어 Python QA 엔지니어 관점에서
[C] Gilded Rose (Python 3, pytest, pytest-cov)
[T] 요구사항을 Python 구현 관점에서 재정리:
  1) 아이템 타입별 비즈니스 규칙 표
    (Normal/Aged Brie/Backstage/Sulfuras/Conjured)
  2) Python 구현 주의점 (== vs in, 정수 경계,
    dataclass 활용 여부)
  3) 예외/경계값 (quality 0~50, sellIn 0/-1/음수)
  4) Conjured 신규 요구사항 (감소율 2배) 명시
  5) pytest 기준 테스트 시나리오 목록
[F] Markdown(표+번호목록).
    requirements_analysis.md 로 저장
```

산출물

이 단계의 포인트

레거시 요구사항 문서를 구현 관점으로 번역

문자열 비교 함정: == vs in 주의

정수 경계·클램프 처리 규칙 명시

Conjured(신규)까지 미리 명세에 포함

→ 이후 모든 단계가 참조하는 밑그림

Mom Test — 재고 담당자 인터뷰로 계약을 발견한다

의견·미래 가정 대신 과거의 구체적 행동·실제 사례·수치만 묻는다. 요구사항 문서만으로는 드러나지 않는 신규 계약(Conjured·quality 경계·GUI 조작)을 찾아낸다.

신규 상품(Conjured) 감가 속도가 기존 로직으로 처리 안 됨	“최근 물건이 유난히 빨리 상하거나 감가 계산이 안 맞은 사례가?”	INV-5
quality 수치가 범위(0~50)를 벗어나는 오류가 실제로 발생	“퀄리티가 0보다 작거나 50 넘게 찍힌 걸 본 적 있나요?”	INV-6
담당자가 매일 하루 경과 처리를 수작업으로 하고 있다	“매일 재고를 어떻게 갱신하시나요? 몇 분 걸리나요?”	AC-1
신규 상품 등록 시 잘못된 값 입력으로 사고가 난 적 있다	“신규 상품 등록 시 값을 잘못 넣어 문제가 된 적 있나요?”	AC-2

증거 수준 부여 & 인터뷰 질문 설계

L0

요구사항 문서에 직접 명시 · 협상 불가능한 제약

L1

담당자 인터뷰(과거 사례)로 확인

L2

근거가 약한 추정

▶ Gemini 프롬프트 — 인터뷰 질문 설계

당신은 Mom Test 코치입니다. 아래 위험 가정을 검증할 인터뷰 질문을 설계해줘.

- 규칙: 의견 · 미래 가정 · 유도 질문 금지. 과거의 구체적 행동 · 실제 사례 · 수치만 묻는다
- 위험 가정: ① Conjured 감가 속도가 다르다 ② quality가 0~50을 벗어난다
③ 하루 경과 처리를 수작업으로 한다 ④ 신규 등록 시 잘못된 값 입력 사고
- 각 가정마다 질문 2개(핵심 + 후속 심화)를 만든다
- 출력: | 위험 가정 | 질문 | 드러낼 계약 후보(INV/E/AC) |
- 나쁜 질문 예와 그것을 고친 좋은 질문 예를 1쌍 덧붙인다

인터뷰 메모 → 테스트 가능한 계약표로 변환

▶ Gemini 프롬프트 — Discovery 에이전트

당신은 Mom Test 방식으로 도메인 규칙을 발견하는 Discovery 에이전트입니다.

목표: 인터뷰 메모에서 테스트 가능한 계약 추출.

Step 1. 각 항목을 증거 수준(L0/L1/L2)으로 분류

Step 2. Invariant / Error / AC 후보 추출

Step 3. ID 부여: INV-N, E-N, AC-N

Step 4. 각 계약을 테스트 가능한 문장으로 변환

나쁜 예: "Conjured는 빨리 상해야 한다."

좋은 예: "sell_in>0 -2, <0 -4, 최소 0."

Step 5. AC는 Given-When-Then으로 작성

Step 6. 출력: |ID|계약|증거|계층|테스트 아이디어|

[인터뷰 메모]

- "Conjured는 공급사 말로 일반의 2배 속도로 상한다. 지난달 폐기량도 유독 많았다."
- "가끔 화면에 quality가 음수거나 50 넘게 나온 걸 본 적 있다. 확실친 않다."
- "매일 아침 엑셀로 하루치 감가를 손으로 계산한다. 버튼 하나로 넘겼으면."
- "신규 등록 시 quality를 60으로 잘못 넣은 적이 있다."

docs/PRD.md — 모든 테스트·구현·커밃의 SSOT

ID	계약 (불변식 / 에러 / 수용 기준)	증거	계층
INV-1	일반: quality 매일 -1, 기한 후 -2, 최소 0	L0	Entity
INV-2	Aged Brie: 매일 +1, 기한 후 +2, 최대 50	L0	Entity
INV-3	Backstage: ≤10 +2, ≤5 +3, 콘서트 후(<0) 0	L0	Entity
INV-4	Sulfuras: quality 80 고정, sell_in 불변	L0	Entity
INV-5 ★	Conjured: sell_in>0 -2, ≤0 -4, 최소 0 (신규)	L1	Entity
INV-6	모든 아이템 $0 \leq \text{quality} \leq 50$ (Sulfuras 제외)	L0/L1	Entity
E-1	Item 클래스/필드 구조 변경 금지	L0	구조
E-2	sell_in 음수 허용 (경계, 예외 아님)	L0	Entity
AC-1	Next Day → 전체 아이템 갱신, 테이블 즉시 반영	L1	Boundary
AC-2	Add Conjured 입력이 0~50 벗어나면 거부·안내	L1	Boundary

이 문서는 SSOT이며 임의 수정 금지 — 변경은 커밃으로만 추적한다.

Test Plan → test_plan.md

▶ Gemini 프롬프트

```
docs/PRD.md @gilded_rose.py
@GildedRoseRequirements.txt @requirements_analysis.md
[P] 시니어 QA 리드입니다.
[C] Python 3, pytest, pytest-cov
[T] 테스트 계획서를 작성해줘.
- pytest 기반 단위 테스트 범위/우선순위
- parametrize 경계값 케이스
  (quality 0,1,49,50 / sellIn 0,-1)
- 예외/특이 케이스 목록
- 커버리지 목표(예: 90%+)와 측정·개선 전략
  pytest --cov=gilded_rose \
          --cov-report=term-missing
[F] Markdown. test_plan.md 로 저장
```

목표

커버리지 측정

- 목표: 라인 커버리지 90% 이상
- term-missing 으로 미커버 라인 확인
- 우선순위 높은 경계값부터 채운다

필수

경계값 표준

quality: 0 · 1 · 49 · 50

sellIn: 0 · -1

이 값들을 parametrize로 5개 타입 전부에 적용

Test Cases → 테스트 코드 (pytest -v Green)

▶ Gemini 프롬프트

```
docs/PRD.md @test_gilded_rose.py
@gilded_rose.py @requirements_analysis.md
[P] 테스트 설계에 강한 시니어 Python QA.
[C] Python 3, pytest
[T] 아이템 타입별 최소 5개 테스트 함수 작성:
  - Normal / Aged Brie / Backstage /
    Sulfuras / Conjured
  - parametrize로 경계값
    (quality 0/50, sellIn 0/-1) 포함
  - Given-When-Then 주석 · assert 검증
  - 이름: test_[타입]_[조건]_[기대결과]
[F] 완성된 테스트 코드.
    pytest -v 가 Green 이 되게 작성
```

규칙

테스트 명명 규칙

`test_[아이템타입]_[조건]_[기대결과]`

예) `test_conjured_after_sellin_`
`degrades_by_four`

Given-When-Then 3단 주석으로 의도를 명시

parametrize로 경계값을 한 함수에 묶어 중복 제거

→ 최종 목표는 `pytest -v` 전부 Green

Defect Detection → defect_list.md

먼저 실행해 실패 로그를 얻는다 → `!pytest -v`

▶ 프롬프트 A — 결함 분석

```
@test_gilded_rose.py @gilded_rose.py
[P] 디버깅·결함 분석에 능한 Python QA.
[T] (여기에 pytest 실패 로그 붙여넣기)
  1) AssertionError 기대/실제 차이 요약
  2) update_quality() 버그 위치 특정
    (파일명:줄번호)
  3) 심각도 분류 (Critical/Major/
    Minor/Info)와 근거
  4) 최소 코드 변경 수정안
    (Item 클래스는 수정 금지)
[F] 수정 diff + Green 확인 절차
```

▶ 프롬프트 B — 결함 문서화

```
[P] QA 리드입니다.
[C] 발견된 테스트 실패/결함을 문서화한다.
[T] 발견 결함을 아래 형식으로
    defect_list.md 에 정리:
    [ID] [Severity] [ItemType]
    [Steps] [Expected] [Actual]
    [Root Cause] [Fix Summary]

[F] Markdown.
    defect_list.md 로 저장

※ 심각도 4단계로 일관되게 분류
```

회귀 안전장치 → test_golden_master.py

▶ Gemini 프롬프트

```
@texttest_fixture.py @gilded_rose.py
[P] 회귀 테스트 (Approval/Golden Master)
    설계 전문가입니다.
[C] Python 3, pytest
[T] texttest_fixture.py 출력 기반
    Golden Master를 설계/구현:
    1) 기준 출력 (golden_master_expected.txt)
        생성/보관 (subprocess.io.StringIO 캡처)
    2) actual 출력과 파일 비교
        (없으면 생성, 있으면 비교)
    3) pytest.ini/conftest.py 에 마킹
    4) CI (GitHub Actions) 에서 자동 실행
[F] test_golden_master.py + 실행 방법
```

Approval 패턴

1

기준 출력 캡처

현재 정상 동작을 파일로 고정

2

없으면 생성

최초 실행 = 승인(approve) 모드

3

있으면 비교

이후 실행마다 actual과 대조

4

불일치 = FAIL

회귀를 자동으로 감지

SOLID & Code Smell → code_quality_report.md

▶ Gemini 프롬프트

```
@gilded_rose.py
[P] 시니어 Python 아키텍트 +
    클린코드 전문가입니다.
[T] update_quality()를 SOLID/
    Code Smell 관점에서 분석:
    - SRP/OCP 위반 지점과 근거
    - Long Method, Magic Number,
      Duplicated Code, 조건 복잡도
    - Python 특화: 타입 힌트 부재,
      문자열 하드코딩, isinstance 미활용
    - PEP8/Pythonic 개선 (전략패턴/dataclass)
    - 리팩토링 우선순위 (1~5)와 이유
[F] Markdown 표. code_quality_report.md
```

Format

보고서 표 형식

- 문제점
- 위반 원칙 / 스멜
- 영향
- 개선 방향
- 우선순위 (1~5)
-
- **주요 스멜 후보:**
- Long Method · Magic Number · 중첩 if · 중복 코드

테스트 Green 유지 → 단계별 체크리스트

▶ Gemini 프롬프트

```
@gilded_rose.py @code_quality_report.md
@requirements_analysis.md
[P] Pythonic 리팩토링 코치입니다.
[C] 제약: Item 수정 금지, quality 0~50,
    테스트 Green에서만 진행
[T] update_quality()를 점진적으로
    리팩토링하는 계획을 제안:
    - 작은 단계(커밋 단위)로 쪼개기
    - 전략 패턴(타입별 updater 분리)
    - dataclass/Protocol 기반 타입 분리
    - 매직 넘버 상수화
      (MAX_QUALITY=50, MIN_QUALITY=0)
    - PEP8 / 타입 힌트 보강
[F] 단계별 체크리스트 + 각 단계 pytest -v
```

리팩토링 황금률

- 테스트가 Green일 때만 손댄다
- 한 커밋 = 한 개선 (작게 쪼갬다)
- 매직 넘버는 명명 상수로
- 각 단계 직후 pytest -v 로 검증

주의

자주 하는 실수

“한 번에 스멜 찾고 고쳐줘” → 분석 없이 코드가 바뀐다.

탐지(읽기전용) → 계획 → 1개 실행 → pytest, 턴을 나눈다.

Defect Management → defect_report.md

▶ Gemini 프롬프트

```
@defect_list.md @requirements_analysis.md
@test_plan.md
[P] QA 리드 엔지니어입니다.
[T] 결함 관리 문서를
    defect_report.md 로 작성:
1) 분류 체계: Severity(4단계) ×
    ItemType(5종) 매트릭스
2) 결함 보고서 템플릿
    (재현/기대/실제/원인/수정/검증)
3) 품질 메트릭 수집 계획
    (통과율·커버리지·발견율, pytest-cov)
4) (선택) GitHub Issues 연동 워크플로우
[F] Markdown. defect_report.md 로 저장
```

Severity 4단계

Critical

즉시 수정 · 릴리스 차단

Major

핵심 기능 결함

Minor

경미한 오작동

Info

개선 제안 · 참고

수집

메트릭

테스트 통과율 · 커버리지 · 단계별 결함 발견율

QA 종합 검토 — 최종 보고서

qa_final_report.md — 지금까지의 모든 산출물을 참조해 활동 전체를 정리한다

▶ Gemini 프롬프트

```
@requirements_analysis.md @code_quality_report.md
@test_plan.md @defect_report.md
@gilded_rose.py @test_gilded_rose.py

[P] QA 리드 엔지니어 관점에서
[T] Gilded Rose QA 활동을 종합 검토해줘.
[F] Markdown 최종 보고서.
    qa_final_report.md 로 저장
```

1

테스트 완료율 / 커버리지

목표 대비 pytest-cov 수치

2

결함 패턴 분석

아이템 타입별 · 심각도별

3

단계별 효과 진단

효과적/개선 필요 단계

4

Best Practice 5가지

다음 레거시 프로젝트용

5

Gemini CLI 활용 효과

시간 단축·조기 발견·커버리지

Conjured 기능 추가 — Item 수정 없이

▶ Gemini 프롬프트

```
@gilded_rose.py @test_gilded_rose.py
@GildedRoseRequirements.txt
Conjured 아이템 기능을 추가해줘.
- 규칙: 매일 quality -2,
  기한(sell_in < 0) 후 -4, 최소 0
- Item 클래스 수정 없이 구현
- 구현 후 pytest -v 실행,
  Green 확인 (실패하면 자동 수정)
- pytest --cov=gilded_rose \
  --cov-report=term-missing 로
  커버리지 확인
```

생성 결과 예시 (Python)

```
def _update_conjured(item):
    rate = 2 if item.sell_in > 0 \
        else 4
    item.quality = max(
        0, item.quality - rate)

# update_quality() 에서 호출:
# if 'Conjured' in item.name:
#     _update_conjured(item)
```

✓ 완료

체크포인트

- pytest -v 가 전부 Green
- Conjured: 매일 -2 · 기한 후 -4 · 최소 0
- Item 클래스(gilded_rose.py)는 그대로

발견 → 계획 → 테스트 → 개선 → 보고

Discovery	0~1 단계	준비 · Mom Test · 계약표 (PRD SSOT)
Plan & Test	2~4 단계	테스트 계획 · 케이스 · 결함 분석
Safety & Quality	5~6 단계	Golden Master · SOLID 진단
Refactor & Report	7~9 단계	리팩토링 · 결함 관리 · 종합 보고
Bonus	보너스	Conjured 추가 · Green 확인

모든 단계는 GEMINI.md (자동 로딩) + P-C-T-F 프롬프트 + @파일 참조 → 산출물.md 저장으로 진행한다.

레거시를 발견하고, 테스트로 방어하고, 리팩토링으로 완성한다.

- ✓ GEMINI.md에 규칙을 못 박으면 모든 프롬프트가 같은 제약을 따른다
- ✓ Mom Test로 문서에 없는 계약(INV-5·AC-2)을 발견하고 PRD를 SSOT로 삼는다
- ✓ Golden Master로 회귀를 막고, 테스트 Green에서만 리팩토링한다
- ✓ Item 수정 없이 Conjured를 추가하는 순간, 계약이 진짜 동작이 된다

수고하셨습니다